

UNIVERSIDADE FEDERAL DE ALFENAS

Curso de Ciência da Computação — Instituto de Ciências Exatas

Disciplina: DCE673 — Programação Funcional

Data: 24/06/2026

1. (20 pt) Faça, em Haskell, a função `f1` que recebe uma `String s` e retorna um `Bool b` que informa se `s` é ou não um palíndromo. Ex: `f1 "ana"` retorna `True`.

```
f1 :: (Eq u) => [u] -> Bool
```

2. (20 pt) Faça, em Haskell, a função `f2` de alta ordem e implementada com *list comprehension* com a seguinte computação: `f2` recebe como parâmetros uma função `f` e uma `[String] l`. Como resultado, deve retornar uma `[(String, Bool)]`, sendo que cada dupla deve ser formada por uma `String` de `l` e o `Bool` que informa se a string é ou não um palíndromo. Ex: `f2 f1 ["ana", "carlos"]` retorna `[("ana", True), ("carlos", False)]`

```
f2 :: (u -> t) -> [u] -> [(u, t)]
```

3. (15 pt) Considere a função `map` do prelude. Assim, implemente a função de alta ordem `f3` a ser passada como parâmetro para a função `map` para que a chamada a seguir tenha a mesma computação de `f2 f1 ["ana", "carlos"]`

```
hugs> map (f3 f1) ["ana", "carlos"]
```

```
f3 :: (u -> t) -> u -> (u, t)
```

4. (20 pt) Faça, em Haskell e com uso de *list comprehension*, a função `f4` que receba uma `[String]` e retorne `[String]` que contém somente as strings que não são palíndromo e que não têm mais que cinco caracteres.

```
f4 :: [String] -> [String]
```

5. (20 pt) Considere que o usuário deseja fazer a mesma computação da função `f4`, mas a partir da chamada que se segue:

```
hugs> filter (f5 f1 length) ["ana", "carlos"]
```

Então, implemente a função `f5` de modo que esta chamada realize a computação desejada.

```
f5 :: ([u] -> Bool) -> ([u] -> Int) -> [u] -> Bool
```

Boa Prova!

eliseu César miguel